Aprendizaje por Refuerzo

Control con Aproximaciones

Luis Daniel Hernández Molinero
Ciencia de la Computación e Inteligencia Artificial
Dept. Ingeniería de la Información y las Comunicaciones

Universidad de Murcia

Máster en Inteligencia Artificial

2025 / 2026

Índice de Contenidos

1. Introducción

2. Control Semi-Gradiente Episódico
   2.1. Control Semi-Gradiente Episódico On-Policy
   Sarsa Semi-Gradiente Episódico
   Sarsa Semi-Gradiente de n-Pasos
   2.2. Control Semi-Gradiente Episódico Off-Policy
   Q-Learning Semi-Gradiente Episódico
   Deep Q-Learning

3. Control Diferencial

4. Referencias

Desarrollo

1. Introducción

2. Control Semi-Gradiente Episódico
   2.1. Control Semi-Gradiente Episódico On-Policy
   Sarsa Semi-Gradiente Episódico
   Sarsa Semi-Gradiente de n-Pasos
   2.2. Control Semi-Gradiente Episódico Off-Policy
   Q-Learning Semi-Gradiente Episódico
   Deep Q-Learning

3. Control Diferencial

4. Referencias

Introducción

- En problemas reales hay **altas dimensiones** en el espacio de estados.
  - **Métodos tabulares son ineficientes** para problemas de gran escala.
  - Necesidad de representar funciones de valor y políticas de manera compacta.

- **En Predicción:**
  - El **objetivo** es construir $\hat{v}(s, \mathbf{w}) \approx v_\pi(s)$ para el valor aproximado del estado s dado el vector de pesos $\mathbf{w} \in \mathbb{R}^d$
  - Y que minimice el **Error Cuadrático Medio de Valor**,

$$\overline{\mathrm{VE}}(\mathbf{w}) \doteq \sum_{s \in S} \mu(s) \left[v_\pi(s) - \hat{v}(s, \mathbf{w}) \right]^2 \text{ para la } \textbf{distribución de estados } \mu(s).$$

  - Lo resolvemos con **gradiente estocástico** tomando ejemplos $S_t \to U_t$:

$$\mathbf{w}_{t+1} \doteq \mathbf{w}_t + \alpha \left[U_t - \hat{v}(S_t, \mathbf{w}_t) \right] \nabla \hat{v}(S_t, \mathbf{w}_t)$$

  - Y por **métodos de semi-gradiente** tomando ejemplos $S_t \to U_t(\mathbf{w})$.

- **En Control:**
  - El **objetivo** es construir un función valor de acción, $\hat{q}(s, a, \mathbf{w}) \approx q_\pi(s, a)$ donde $\mathbf{w} \in \mathbb{R}^d$ es un vector de pesos.
  - Ahora tomaremos ejemplos de la forma $S_t, A_t \to U_t$.
  - Veremos algunas alternativas.

Desarrollo

1. Introducción

2. Control Semi-Gradiente Episódico
   2.1. Control Semi-Gradiente Episódico On-Policy
   Sarsa Semi-Gradiente Episódico
   Sarsa Semi-Gradiente de n-Pasos
   2.2. Control Semi-Gradiente Episódico Off-Policy
   Q-Learning Semi-Gradiente Episódico
   Deep Q-Learning

3. Control Diferencial

4. Referencias

Profundizamos en ...

1. Introducción

2. Control Semi-Gradiente Episódico
   2.1. Control Semi-Gradiente Episódico On-Policy
   Sarsa Semi-Gradiente Episódico
   Sarsa Semi-Gradiente de n-Pasos
   2.2. Control Semi-Gradiente Episódico Off-Policy
   Q-Learning Semi-Gradiente Episódico
   Deep Q-Learning

3. Control Diferencial

4. Referencias

Con más detalle ..

1. Introducción

2. Control Semi-Gradiente Episódico
   2.1. Control Semi-Gradiente Episódico On-Policy
   Sarsa Semi-Gradiente Episódico
   Sarsa Semi-Gradiente de n-Pasos
   2.2. Control Semi-Gradiente Episódico Off-Policy
   Q-Learning Semi-Gradiente Episódico
   Deep Q-Learning

3. Control Diferencial

4. Referencias

Control Semi-Gradiente Episódico

- La actualización general de **descenso de gradiente estocástico** para la predicción del valor de acción es:

$$\mathbf{w}_{t+1} \doteq \mathbf{w}_t + \alpha\left[U_t - \hat{q}(S_t, A_t, \mathbf{w}_t)\right]\nabla\hat{q}(S_t, A_t, \mathbf{w}_t).$$

  tomando ejemplos de la forma $S_t, A_t \rightarrow U_t$.

- Episódico **gradiente Monte Carlo**: $U_t = G_t$
- Episódico **semi-gradiente Sarsa de un paso**: $U_t = R_{t+1} + \gamma\hat{q}(S_{t+1}, A_{t+1}, \mathbf{w}_t)$
  - Se puede generalizar a n-pasos.
- Seguimos aplicando el enfoque de **Iteración de Políticas Generalizada**. Si es on-policy:
  - Debemos hacer una estimación del valor $\hat{q}_\pi(S_t, a, \mathbf{w}_t)$
  - **Mejoramos la política**:
    - Para cada posible acción en un estado S_t seleccionamos la acción greedy: $A_{t+1} = \arg\max_a \hat{q}(S_{t+1}, a, \mathbf{w}_t)$.
    - La mejora de la política se consigue con una política soft sobre la política greedy (p.e. una ϵ-greedy).

Sarsa semi-gradiente episódico para estimar $\hat{q} \approx q_0$

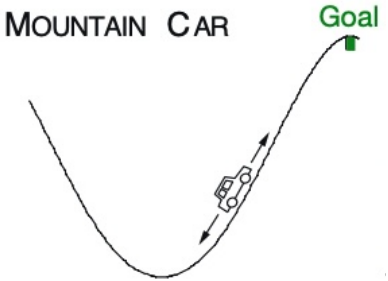
Episodic Semi-gradient Sarsa for Estimating $\hat{q} \approx q_*$

Input: a differentiable action-value function parameterization $\hat{q} : \mathcal{S} \times \mathcal{A} \times \mathbb{R}^d \to \mathbb{R}$
Algorithm parameters: step size $\alpha > 0$, small $\varepsilon > 0$
Initialize value-function weights $\mathbf{w} \in \mathbb{R}^d$ arbitrarily (e.g., $\mathbf{w} = \mathbf{0}$)

Loop for each episode:
  $S, A \leftarrow$ initial state and action of episode (e.g., ε-greedy)
  Loop for each step of episode:
    Take action A, observe R, S'
    If S' is terminal:
      $\mathbf{w} \leftarrow \mathbf{w} + \alpha\big[R - \hat{q}(S, A, \mathbf{w})\big]\nabla\hat{q}(S, A, \mathbf{w})$
      Go to next episode
    Choose A' as a function of $\hat{q}(S', \cdot, \mathbf{w})$ (e.g., ε-greedy)
    $\mathbf{w} \leftarrow \mathbf{w} + \alpha\big[R + \gamma\hat{q}(S', A', \mathbf{w}) - \hat{q}(S, A, \mathbf{w})\big]\nabla\hat{q}(S, A, \mathbf{w})$
    $S \leftarrow S'$
    $A \leftarrow A'$

Ejemplo: *Mountain Car Task* – I

- **La única forma de llegar** es acelerar al máximo desde la pendiente izquierda y con la inercia llegar al objetivo.
- La **recompensa** en este problema es de -1 en todos los pasos de tiempo hasta que el coche alcance su posición objetivo en la cima de la montaña, lo que termina el episodio.
- Hay **tres acciones** posibles:
  - Aceleración completa hacia adelante ($+1$).
  - Aceleración completa hacia atrás (-1).
  - Sin aceleración (0).
- Cada episodio **comienza** desde una posición aleatoria $x_t \in [-0{,}6, -0{,}4]$ y con velocidad cero.
- El coche se mueve de acuerdo con una **física simplificada**.



$$x_{t+1} \doteq \text{bound}\,(x_t + \dot{x}_t + 1) \Rightarrow -1{,}2 \leq x_{t+1} \leq 0{,}5$$

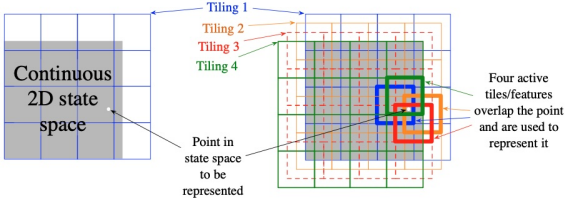
$$\dot{x}_{t+1} \doteq \text{bound}\,(\dot{x}_t + 0{,}001 A_t - 0{,}0025\cos(3x_t)) \Rightarrow -0{,}07 \leq \dot{x}_{t+1} \leq 0{,}07$$

Ejemplo: *Mountain Car Task* – II

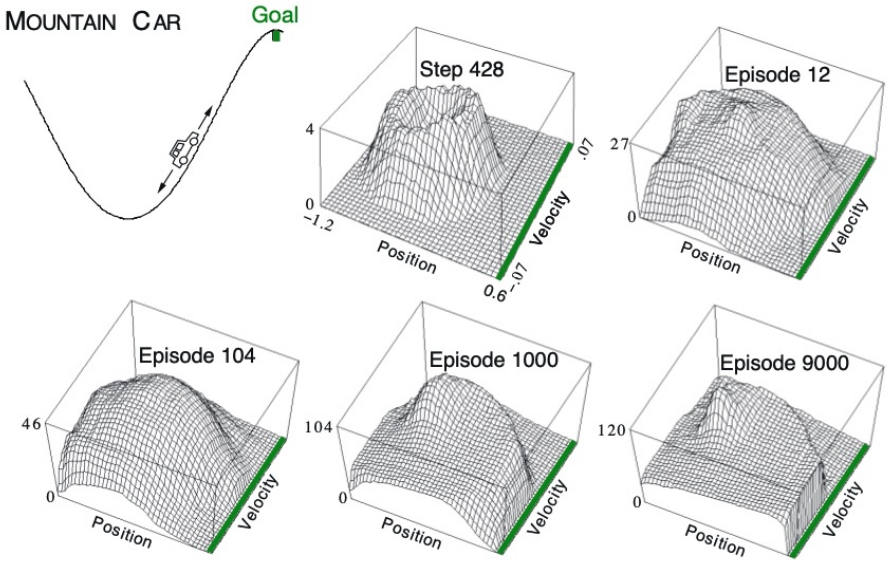
- El **espacio de estados es continuo** y bidimensional (posición y velocidad).
- Se utilizaron **8 recubrimientos** (**tilings**), con cada recubrimiento cubriendo 1/8 de la distancia limitada en cada dimensión, y con desplazamientos asimétricos.
- Los vectores de características $x(s, a)$, creados mediante **codificación por recubrimientos** (**tile coding**), se combinaron **linealmente** con el vector de parámetros para aproximar la función de valor–acción:

$$\hat{q}(s, a, \mathbf{w}) = \mathbf{w}^\top \mathbf{x}(s, a) = \sum_{i=1}^{d} w_i \cdot x_i(s, a), \text{ donde:}$$

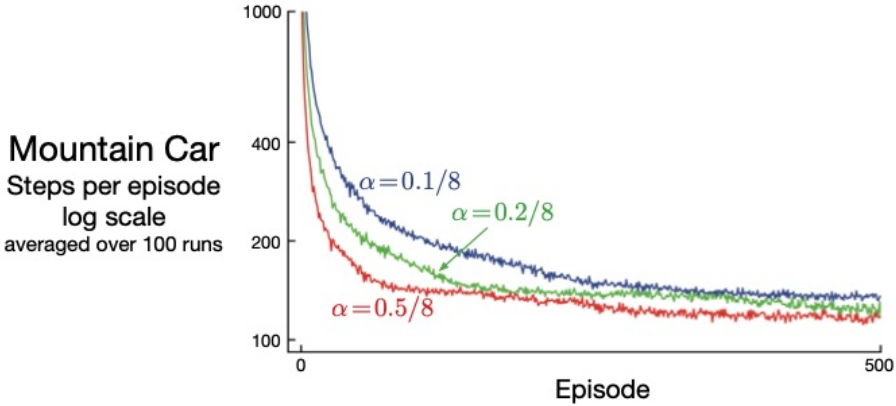
  - $\hat{q}(s, a, \mathbf{w})$: Valor aproximado de la función $Q(s, a)$.
  - $\mathbf{w}$: Vector de parámetros.
  - $x(s, a)$: Vector de características generado por los recubrimientos.
  - d: Número de características activas.



Ejemplo: *Mountain Car Task* – III
La función de costo (maximización de $\hat{q}(s, a, \mathbf{w})$) aprendida



Ejemplo: *Mountain Car Task* - y IV
Curvas de Aprendizaje



- Curvas de aprendizaje para el coche en la montaña usando el método semi-gradiente Sarsa con una aproximación de función basada en *tile-coding* y selección de acción ϵ-codiciosa.

Con más detalle ..

1. Introducción

2. Control Semi-Gradiente Episódico
   2.1. Control Semi-Gradiente Episódico On-Policy
   Sarsa Semi-Gradiente Episódico
   Sarsa Semi-Gradiente de n-Pasos
   2.2. Control Semi-Gradiente Episódico Off-Policy
   Q-Learning Semi-Gradiente Episódico
   Deep Q-Learning

3. Control Diferencial

4. Referencias

Sarsa de n-pasos Semi-Gradiente

- Podemos obtener una **versión de n pasos** del Sarsa semi-gradiente episódico utilizando un retorno de n pasos como el objetivo de actualización en la ecuación de actualización de Sarsa semi-gradiente.
- El **retorno de n pasos** se generaliza desde su forma tabular a una aproximación de funciones:

$$G_{t:t+n} \doteq \begin{cases} R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^{n-1} R_{t+n} + \gamma^n \hat{q}(S_{t+n}, A_{t+n}, \mathbf{w}_{t+n-1}) & \text{si } t+n < T \\ G_t & \text{si } t+n \geq T \end{cases}$$

- La **ecuación de actualización** de n pasos es definir $U_t \doteq G_{t:t+n}$:

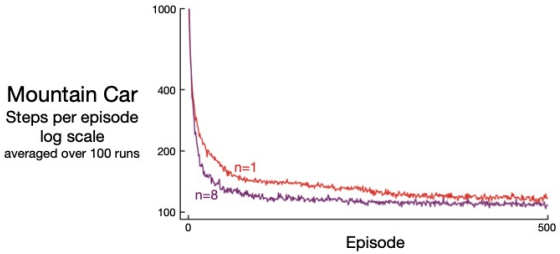
$$\mathbf{w}_{t+n} \doteq \mathbf{w}_{t+n-1} + \alpha \left[G_{t:t+n} - \hat{q}(S_t, A_t, \mathbf{w}_{t+n-1}) \right] \nabla \hat{q}(S_t, A_t, \mathbf{w}_{t+n-1}), \quad 0 \leq t < T$$

Episodic semi-gradient n-step Sarsa for estimating $\hat{q} \approx q_*$ or q_π

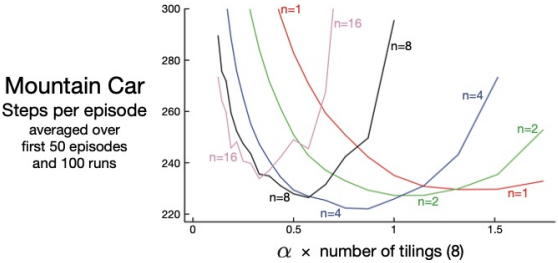
Input: a differentiable action-value function parameterization $\hat{q} : \mathcal{S} \times \mathcal{A} \times \mathbb{R}^d \to \mathbb{R}$
Input: a policy π (if estimating q_π)
Algorithm parameters: step size $\alpha > 0$, small $\varepsilon > 0$, a positive integer n
Initialize value-function weights $\mathbf{w} \in \mathbb{R}^d$ arbitrarily (e.g., $\mathbf{w} = \mathbf{0}$)
All store and access operations (S_t, A_t, and R_t) can take their index mod $n+1$

Loop for each episode:
  Initialize and store $S_0 \neq$ terminal
  Select and store an action $A_0 \sim \pi(\cdot|S_0)$ or ε-greedy wrt $\hat{q}(S_0, \cdot, \mathbf{w})$
  $T \leftarrow \infty$
  Loop for $t = 0, 1, 2, \ldots$:
  | If $t < T$, then:
  | Take action A_t
  | Observe and store the next reward as R_{t+1} and the next state as S_{t+1}
  | If S_{t+1} is terminal, then:
  | $T \leftarrow t + 1$
  | else:
  | Select and store $A_{t+1} \sim \pi(\cdot|S_{t+1})$ or ε-greedy wrt $\hat{q}(S_{t+1}, \cdot, \mathbf{w})$
  | $\tau \leftarrow t - n + 1$ (τ is the time whose estimate is being updated)
  | If $\tau \geq 0$:
  | $G \leftarrow \sum_{i=\tau+1}^{\min(\tau+n,T)} \gamma^{i-\tau-1} R_i$
  | If $\tau + n < T$, then $G \leftarrow G + \gamma^n \hat{q}(S_{\tau+n}, A_{\tau+n}, \mathbf{w})$ $(G_{\tau:\tau+n})$
  | $\mathbf{w} \leftarrow \mathbf{w} + \alpha \left[G - \hat{q}(S_\tau, A_\tau, \mathbf{w})\right] \nabla \hat{q}(S_\tau, A_\tau, \mathbf{w})$
  Until $\tau = T - 1$

Ejemplo: *Mountain Car Task*
Sarsa n-Pasos Semi Gradiente Episódico



- Rendimiento de 1 paso y 8 pasos del método semi-gradiente de Sarsa en la tarea Mountain Car.
- Tamaños de paso: $\alpha = 0{,}5/8$ para $n = 1$ y $\alpha = 0{,}3/8$ para $n = 8$.



- Efecto de α y n.
- Un nivel intermedio de bootstrap ($n = 4$) tuvo el mejor rendimiento.
- Los errores estándar variaron de 0.5 para $n = 1$ hasta 16 para $n = 16$.

$$\text{Error Estándar} = \frac{\text{Desviación estándar de la muestra}}{\text{raíz cuadrada del número de observaciones}}$$

Profundizamos en ...

1. Introducción

2. Control Semi-Gradiente Episódico
   2.1. Control Semi-Gradiente Episódico On-Policy
   Sarsa Semi-Gradiente Episódico
   Sarsa Semi-Gradiente de n-Pasos
   2.2. Control Semi-Gradiente Episódico Off-Policy
   Q-Learning Semi-Gradiente Episódico
   Deep Q-Learning

3. Control Diferencial

4. Referencias

Con más detalle ..

1. Introducción

2. Control Semi-Gradiente Episódico
   2.1. Control Semi-Gradiente Episódico On-Policy
   Sarsa Semi-Gradiente Episódico
   Sarsa Semi-Gradiente de n-Pasos
   2.2. Control Semi-Gradiente Episódico Off-Policy
   Q-Learning Semi-Gradiente Episódico
   Deep Q-Learning

3. Control Diferencial

4. Referencias

Q-Learning Semi-Gradiente

- **Recuerda**: Q-learning es un método **off-policy**, mientras que Sarsa es **on-policy**.
- **Métodos de semi-gradiente:**

$$\mathbf{w}_{t+1} \doteq \mathbf{w}_t + \alpha[\underbrace{U_t(\mathbf{w}_t)}_{\text{Target}} - \hat{q}(S_t, A_t, \mathbf{w}_t)]\nabla \hat{q}(S_t, A_t, \mathbf{w}_t)$$

  - Se corresponde a la actualización en un paso del Método del Descenso de Gradiente aplicada a la función Error Cuadrático Medio.
- Episódico **semi-gradiente Sarsa de un paso** (on-policy):

$$U_t(\mathbf{w}_t) = R_{t+1} + \gamma\, \hat{q}(S_{t+1}, A_{t+1}, \mathbf{w}_t)$$

- Episódico **semi-gradiente Q-learning de un paso** (off-policy):

$$U_t(\mathbf{w}_t) \doteq R_{t+1} + \gamma \max_{a'} \hat{q}(S_{t+1}, a', \mathbf{w}_t) \qquad \text{(Q-Target)}$$

$$\mathbf{w}_{t+1} \doteq \mathbf{w}_t + \alpha \left[\overbrace{\underbrace{R_{t+1} + \gamma \max_{a'} \hat{q}(S_{t+1}, a', \mathbf{w}^-)}_{\text{Q-Target}} - \hat{q}(S_t, A_t \mathbf{w}_t)}^{\text{Q-Pérdida}} \right] \nabla \hat{q}(S_t, A_t, \mathbf{w}_t)$$

Con más detalle ..

1. Introducción

2. Control Semi-Gradiente Episódico
   2.1. Control Semi-Gradiente Episódico On-Policy
   Sarsa Semi-Gradiente Episódico
   Sarsa Semi-Gradiente de n-Pasos
   2.2. Control Semi-Gradiente Episódico Off-Policy
   Q-Learning Semi-Gradiente Episódico
   Deep Q-Learning

3. Control Diferencial

4. Referencias

Deep Q-Learning (DQN)
Ha nacido un estrella (Mnih et al., 2013) (Mnih et al., 2015)

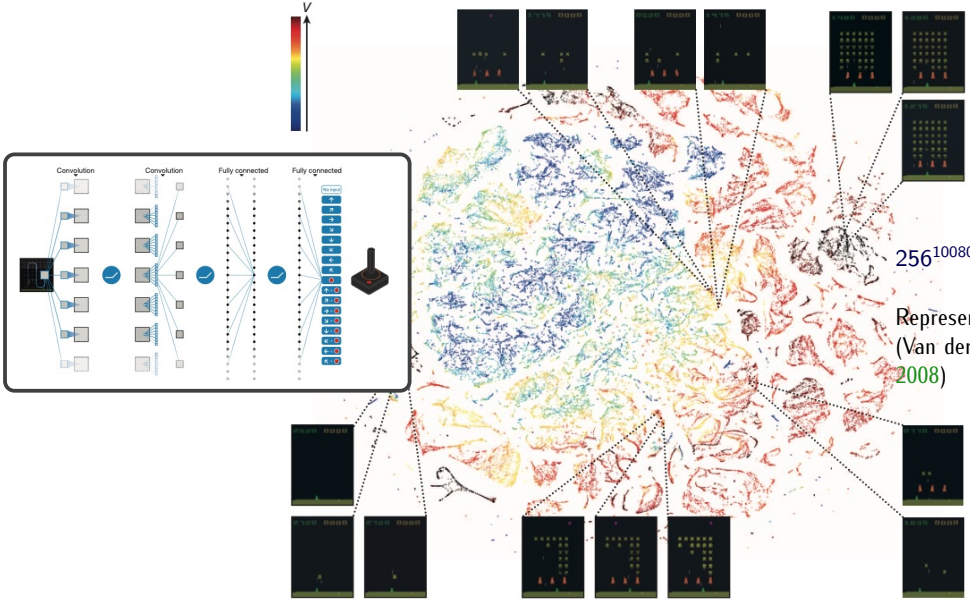
- **Deep Q-Learning (DQN)** es una extensión de Q-Learning clásico que utiliza **redes neuronales profundas** para aproximar $q(s, a)$ con una red neuronal que:
  - Toma un estado s como entrada:
    - Un vector (en entornos simples), una imagen (en videojuegos o tareas visuales).
  - La salida produce un vector de valores $q(s, a)$ para todas las acciones posibles a.
- La función $\hat{q}(s, a; \mathbf{w})$ de aproximación es una red neurona donde $\mathbf{w}$ son los parámetros de la red. .
- **Ejemplo** para videojuegos (como Atari):
  - Entrada: Imágenes del juego procesada en blanco y negro.
  - Salida: Valores $q(s, a)$ para acciones como mover a la izquierda, derecha, o disparar.
- **Características de DQN:**
  - La red se entrena para **minimizar el error cuadrático medio** (MSE) entre la salida actual y el objetivo:
    $$L(\mathbf{w}) = \mathbb{E}_{(s,a,r,s')}\left[(y - \hat{q}(s, a; \mathbf{w}))^2\right],$$
    donde:
    - $\hat{q}(s, a; \mathbf{w})$ es la estimación actual.
    - **El target es una copia congelada** de la red principal.
      - $y = r + \gamma \max_{a'} \hat{q}(s', a'; \mathbf{w}_{\text{target}})$.
      - $\mathbf{w}_{\text{target}}$ son los parámetros de una red separada (*target network*).
  - **Aplica Replay Buffer** (muestreo)

Algoritmo Deep Q-Learning *con Experience Replay*

Algorithm 1 Deep Q-learning with Experience Replay

Initialize replay memory $\mathcal{D}$ to capacity N
Initialize action-value function Q with random weights
for episode $= 1, M$ **do**
  Initialise sequence $s_1 = \{x_1\}$ and preprocessed sequenced $\phi_1 = \phi(s_1)$
  for $t = 1, T$ **do**
    With probability ϵ select a random action a_t
    otherwise select $a_t = \max_a Q^*(\phi(s_t), a; \theta)$
    Execute action a_t in emulator and observe reward r_t and image x_{t+1}
    Set $s_{t+1} = s_t, a_t, x_{t+1}$ and preprocess $\phi_{t+1} = \phi(s_{t+1})$
    Store transition $(\phi_t, a_t, r_t, \phi_{t+1})$ in $\mathcal{D}$
    Sample random minibatch of transitions $(\phi_j, a_j, r_j, \phi_{j+1})$ from $\mathcal{D}$
    Set $y_j = \begin{cases} r_j & \text{for terminal } \phi_{j+1} \\ r_j + \gamma \max_{a'} Q(\phi_{j+1}, a'; \theta) & \text{for non-terminal } \phi_{j+1} \end{cases}$
    Perform a gradient descent step on $(y_j - Q(\phi_j, a_j; \theta))^2$ according to equation 3
  end for
end for

Deep Q-Learning con Juegos de ATARI



256^{100800} **estados**

Representación usando t-SNE (Van der Maaten & Hinton, 2008)

Algoritmo Double Deep Q-Learning
(Van Hasselt et al., 2016)

- En vez de $y = r + \gamma \max_{a'} \hat{q}(s', a', \mathbf{w}^-)$, usa $y = r + \gamma\, \hat{q}(s', \arg\max_{a'} \hat{q}(s', a', \mathbf{w}), \mathbf{w}^-)$

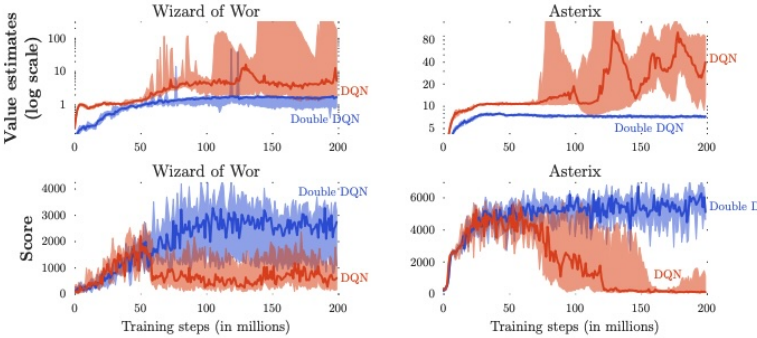
- **DQN presenta sobreestimación.**
  - El $\max_a$ magnifica el error.
  - Conduce a políticas subóptimas
- **Aprendizaje doble.**
  - Es insesgado
  - **Red Principal** (online network)

    $$\hat{q}(s, a, \mathbf{w})$$

    selecciona las acciones a'
  - **Red objetivo** (target network)

    $$\hat{q}(s, a, \mathbf{w}^-)$$

    evalúa la acción seleccionada.



- Doble DQN evalúa por debajo de DQN (que soebreestima)
- Doble DQN consigue más recompensas.

Desarrollo

1. Introducción

2. Control Semi-Gradiente Episódico
   2.1. Control Semi-Gradiente Episódico On-Policy
   Sarsa Semi-Gradiente Episódico
   Sarsa Semi-Gradiente de n-Pasos
   2.2. Control Semi-Gradiente Episódico Off-Policy
   Q-Learning Semi-Gradiente Episódico
   Deep Q-Learning

3. Control Diferencial

4. Referencias

Recompensa Promedio
Una Nueva Configuración para Tareas Continuas

- En el **caso continuo** el concepto de horizonte temporal (y por tanto de γ) pierde relevancia.
- Un **ejemplo** intuitivo. La recompensa de una máquina dron.
  - **Caso episódico:** Con descuento γ, prioriza las ganancias tempranas. $G_t = R_{t+1} + \gamma R_{t+2} + \ldots + \gamma^{T-t-1} R_T$
  - **Caso continuo:** Es más correcto considerar el **promedio de recompensas** en el tiempo. $r(\pi) = \lim_{T\to\infty} \frac{1}{T} \sum_{t=1}^{T} R_t$
- **Recompensa promedio de una política**: es el promedio de las recompensas cuando se sigue una política en el tiempo:

$$
\begin{aligned}
r(\pi) &\doteq \lim_{h\to\infty} \frac{1}{h} \sum_{t=1}^{h} \mathbb{E}[R_t \mid S_0, A_{0:t-1} \sim \pi], \qquad \text{(ergocidad y estacionaridad*)} \\
&= \lim_{t\to\infty} \mathbb{E}[R_t \mid S_0, A_{0:t-1} \sim \pi] = \sum_s \mu_\pi(s) \sum_a \pi(a \mid s) \sum_{s',r} p(s', r \mid s, a) r,
\end{aligned}
$$

- **Cuando el entorno es estacionario***, la recompensa esperada en cualquier momento t converge a la recompensa promedio a largo plazo.
  - La estacionariedad implica que existe una **distribución de estados estacionaria**, denotada por $\mu_\pi(s)$, tal que:

$$
\mu_\pi(s) \doteq \lim_{t\to\infty} \Pr(S_t = s \mid A_{0:t-1} \sim \pi) \qquad \mu_\pi(s') = \sum_s \mu_\pi(s) \sum_a \pi(a|s) p(s'|s, a).
$$

  - Se asume que μ_π existe para cualquier política π y es independiente de S_0.
  - La **ergodicidad** es suficiente para garantizar la existencia de los límites en las ecuaciones anteriores.

Relación entre Recompensas

- Con la aproximación de funciones en el caso continuo **se pierde el teorema de mejora** de políticas:

  Si se construye π' tal que: $q_\pi(s, \pi'(s)) \geq v_\pi(s), \quad \forall s \in \mathcal{S}$, entonces: $v_{\pi'}(s) \geq v_\pi(s), \quad \forall s \in \mathcal{S}$.
- **Como alternativa** podemos usar la recompensa promedio: $\pi' \succ \pi \Leftrightarrow r(\pi') \geq r(\pi)$
- **Exite una relación** entre los retornos descontados y el retorno promedio:
  - $G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots$
  - Promedio de los retornos descontados: $\lim_{T\to\infty} \frac{1}{T} \sum_{t=1}^{T} G_t$
  - Retorno promedio: $r(\pi) = \lim_{T\to\infty} \frac{1}{T} \sum_{t=1}^{T} R_t$
  - En un sistema estacionario bajo una política π: $\lim_{T\to\infty} \frac{1}{T} \sum_{t=1}^{T} G_t = \frac{r(\pi)}{1-\gamma}$
- **En un sistema estacionario**,
  - El promedio de los retornos descontados contiene la misma información que el retorno promedio, pero ajustada por $\frac{1}{1-\gamma}$.
  - **Si optimizamos el valor descontado sobre la distribución on-policy, el efecto será idéntico al optimizar la recompensa promedio descontada**

Recomponiendo Todo con Recompensas Promedio – I

- **Retorno diferencial:** En la configuración de recompensa promedio, los retornos se definen en términos de diferencias entre las recompensas y la recompensa promedio:

$$G_t \doteq R_{t+1} - r(\pi) + R_{t+2} - r(\pi) + R_{t+3} - r(\pi) + \cdots = \sum_{k=0}^{\infty} (R_{t+k+1} - r(\pi))$$

- **Funciones de valor diferenciales (Ecuaciones de Bellman):** Eliminamos todos los γ y reemplazamos todas las recompensas por la diferencia entre la recompensa y la recompensa promedio verdadera:

$$v_\pi(s) = \sum_a \pi(a|s) \sum_{r,s'} p(s', r|s, a) \left[r - r(\pi) + v_\pi(s') \right].$$

$$q_\pi(s, a) = \sum_{r,s'} p(s', r|s, a) \left[r - r(\pi) + \sum_{a'} \pi(a'|s') q_\pi(s', a') \right].$$

$$v_*(s) = \max_\pi \sum_{r,s'} p(s', r|s, a) \left[r - \max_\pi r(\pi) + v_*(s') \right].$$

$$q_*(s, a) = \sum_{r,s'} p(s', r|s, a) \left[r - \max_\pi r(\pi) + \max_{a'} q_*(s', a') \right].$$

Recomponiendo Todo con Recompensas Promedio – II

- También existe una **forma diferencial de los dos errores de TD**:
  - Error para la función valor estado
$$\delta_t \doteq R_{t+1} - \overline{R}_t + \hat{v}(S_{t+1}, w_t) - \hat{v}(S_t, \mathbf{w}_t) \qquad (1)$$
  - Error para la función valor estado-acción:
$$\delta_t \doteq R_{t+1} - \overline{R}_t + \hat{q}(S_{t+1}, A_{t+1}, w_t) - \hat{q}(S_t, A_t, \mathbf{w}_t) \qquad (2)$$

  donde $\overline{R}_t$ es una estimación de la recompensa promedio $r(\pi)$.

- Con estas definiciones alternativas, la mayoría de nuestros algoritmos y muchos resultados teóricos se pueden aplicar al enfoque de recompensa promedio sin cambios.

- Por ejemplo, la versión **recompensa promedio de Sarsa semi-gradiente**
$$\mathbf{w}_{t+1} \doteq \mathbf{w}_t + \alpha \delta_t \nabla \hat{q}(S_t, A_t, \mathbf{w}_t)$$
  con δ_t dado por (2).

Sarsa n-pasos Semi-Gradiente Diferencial $\hat{q} \approx q_\pi$

Differential semi-gradient n-step Sarsa for estimating $\hat{q} \approx q_\pi$ or q_*

Input: a differentiable function $\hat{q} : \mathcal{S} \times \mathcal{A} \times \mathbb{R}^d \to \mathbb{R}$, a policy π
Initialize value-function weights $\mathbf{w} \in \mathbb{R}^d$ arbitrarily (e.g., $\mathbf{w} = \mathbf{0}$)
Initialize average-reward estimate $\bar{R} \in \mathbb{R}$ arbitrarily (e.g., $\bar{R} = 0$)
Algorithm parameters: step sizes $\alpha, \beta > 0$, small $\varepsilon > 0$, a positive integer n
All store and access operations (S_t, A_t, and R_t) can take their index mod $n+1$

Initialize and store S_0 and A_0
Loop for each step, $t = 0, 1, 2, \ldots$:
  Take action A_t
  Observe and store the next reward as R_{t+1} and the next state as S_{t+1}
  Select and store an action $A_{t+1} \sim \pi(\cdot|S_{t+1})$, or ε-greedy wrt $\hat{q}(S_{t+1}, \cdot, \mathbf{w})$
  $\tau \leftarrow t - n + 1$ (τ is the time whose estimate is being updated)
  If $\tau \geq 0$:
    $\delta \leftarrow \sum_{i=\tau+1}^{\tau+n} (R_i - \bar{R}) + \hat{q}(S_{\tau+n}, A_{\tau+n}, \mathbf{w}) - \hat{q}(S_\tau, A_\tau, \mathbf{w})$
    $\bar{R} \leftarrow \bar{R} + \beta\delta$
    $\mathbf{w} \leftarrow \mathbf{w} + \alpha\delta\nabla\hat{q}(S_\tau, A_\tau, \mathbf{w})$

Desarrollo

1. Introducción

2. Control Semi-Gradiente Episódico
   2.1. Control Semi-Gradiente Episódico On-Policy
   Sarsa Semi-Gradiente Episódico
   Sarsa Semi-Gradiente de n-Pasos
   2.2. Control Semi-Gradiente Episódico Off-Policy
   Q-Learning Semi-Gradiente Episódico
   Deep Q-Learning

3. Control Diferencial

4. Referencias

Referencias – I

- Bellemare, M. G., Naddaf, Y., Veness, J., & Bowling, M. (2013).The Arcade Learning Environment: An Evaluation Platform for General Agents. *Journal of Artificial Intelligence Research, 47*, 253–279. https://github.com/Farama-Foundation/Arcade-Learning-Environment?tab=readme-ov-file
- Ecoffet, A., Huizinga, J., Lehman, J., Stanley, K. O., & Clune, J. (2021).First return, then explore. *Nature, 590*(7847), 580–586. https://doi.org/10.1038/s41586-020-03157-9
- Lapan, M. (2018). *Deep Reinforcement Learning Hands-On*. Packt Publishing. https://github.com/PacktPublishing/Deep-Reinforcement-Learning-Hands-On-Third-Edition
- Mnih, V., Kavukcuoglu, K., Silver, D., Graves, A., Antonoglou, I., Wierstra, D., & Riedmiller, M. (2013).Playing Atari with Deep Reinforcement Learning. https://www.cs.toronto.edu/~vmnih/docs/dqn.pdf
- Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., Graves, A., Riedmiller, M., Fidjeland, A. K., Ostrovski, G., Petersen, S., Beattie, C., Sadik, A., Antonoglou, I., King, H., Kumaran, D., Wierstra, D., Legg, S., & Hassabis, D. (2015).Human-level control through deep reinforcement learning. *Nature, 518*(7540), 529–533. https://doi.org/10.1038/nature14236
- Silver, D. (2015). Lectures on Reinforcement Learning. https://www.davidsilver.uk/teaching/
- Sutton, R., & Barto, A. (2018). *Reinforcement Learning, second edition: An Introduction*. MIT Press. https://web.stanford.edu/class/psych209/Readings/SuttonBartoIPRLBook2ndEd.pdf

Referencias – II

- Van der Maaten, L., & Hinton, G. (2008).Visualizing high-dimensional data using t-SNE. *Journal of Machine Learning Research, 9*(Nov), 2579–2605. http://www.jmlr.org/papers/volume9/vandermaaten08a/vandermaaten08a.pdf
- Van Hasselt, H., Guez, A., & Silver, D. (2016).Deep Reinforcement Learning with Double Q-learning. *Proceedings of the Thirtieth AAAI Conference on Artificial Intelligence*, 2094–2100. https://arxiv.org/abs/1509.06461
- Wang, Z., Schaul, T., Hessel, M., Van Hasselt, H., Lanctot, M., & De Freitas, N. (2016).Dueling Network Architectures for Deep Reinforcement Learning. *Proceedings of the 33rd International Conference on Machine Learning (ICML), 48*, 1995–2003. https://proceedings.mlr.press/v48/wangf16.html
- Whitaker, T. (2023). Human-level control through deep reinforcement learning – GitHub repository.